

RAG-Solidity: Retrieval-Augmented Generation for Automated Documentation of Smart Contracts

*A Master of Technology Thesis Submitted
in Partial Fulfillment of the Requirements
for the Degree of*

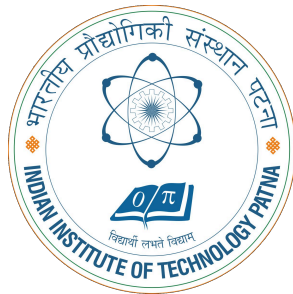
Master of Technology
in Artificial Intelligence

by

Rishav Anand
(2411AI37)

under the guidance of

Dr. Raju Halder
Associate Professor



to the

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY PATNA
PATNA – 800013, BIHAR**

May 2026

THESIS CERTIFICATE

This is to certify that the thesis titled **RAG-Solidity: Retrieval-Augmented Generation for Automated Documentation of Smart Contracts**, submitted by **Rishav Anand** (2411AI37), to the Indian Institute of Technology Patna, for the award of the degree of **Master of Technology in Artificial Intelligence**, is a bona fide record of the research work done by him under our supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.



Dr. Raju Halder

Supervisor

Associate Professor

Department of Computer Science and Engineering

IIT-Patna, 800 013

Place: Patna

Date: _____

DECLARATION

I certify that

The work contained in this thesis is original and has been done by me under the guidance of my supervisor.

The work has not been submitted to any other Institute for any degree or diploma.

I have followed the guidelines provided by the Institute in preparing the thesis.

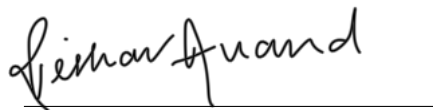
I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

Wherever I have used materials (data, theoretical analysis, and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

Name of the student

Signature of the student

Rishav Anand

A handwritten signature in black ink, reading "Rishav Anand", is written over a horizontal line.

Roll No. 2411AI37

ACKNOWLEDGEMENTS

I am deeply grateful to my thesis supervisor, **Dr. Raju Halder**, for his invaluable guidance, constant encouragement, and critical feedback throughout this work. His insights shaped the research direction in ways I had not anticipated.

I sincerely thank the faculty and staff of the Department of Computer Science and Engineering for providing an excellent academic environment. The open-source contributions of the Salesforce Research team (CodeT5), Hugging Face, and the OpenZeppelin project were instrumental in making this work possible.

I owe a special debt of gratitude to **Md. Tauseef Alam**, Ph.D. Scholar, Department of Computer Science and Engineering, IIT Patna, who helped me enormously throughout this project. His constant encouragement, thoughtful suggestions, and willingness to discuss every technical difficulty made a decisive difference to this thesis. I am deeply thankful for his invaluable support.

I also thank my colleagues for their generous help and my family for their unwavering support and patience.

Rishav Anand

Roll No. 2411AI37

Department of Computer Science and Engineering

Indian Institute of Technology Patna

May 2026

Abstract

Smart contracts written in Solidity form the backbone of the decentralised finance ecosystem on Ethereum. Once deployed, their bytecode cannot be modified, so a poorly documented function can mislead auditors, invite exploits, and ultimately cause irreversible financial loss. Despite this risk, most publicly verified contracts on Etherscan carry incomplete or no NatSpec comments, largely because writing good documentation demands both Solidity knowledge and clear technical prose — a combination that is uncommon.

This thesis describes **RAG-Solidity**, a system built to generate short English comments for Solidity functions automatically. The core idea is simple: before asking a language model to describe a function, first retrieve a handful of similar, already-documented functions from a training corpus and show them to the model as worked examples. Technically, the system uses the encoder of CodeT5-base to embed each function into a 768-dimensional vector, retrieves the top- K nearest neighbours by cosine similarity, and builds a structured prompt that the CodeT5 sequence-to-sequence model then decodes using beam search. A lightweight post-processing step catches any output that looks like code rather than prose and substitutes the closest neighbour’s comment instead.

Experiments on a 1,200-pair Solidity dataset show that retrieval augmentation consistently helps. RAG-Solidity scores BLEU **34.7**, ROUGE-L **47.3**, METEOR **27.8**, and BERTScore **0.87** on the held-out test split, compared with BLEU 28.9 for plain CodeT5 fine-tuning — a relative improvement of roughly **20%** with no increase in model size or inference hardware requirements.

Keywords: retrieval-augmented generation, smart contracts, Solidity, code documentation, CodeT5, blockchain, sequence-to-sequence models.

Contents

Thesis Certificate	i
Declaration	ii
Acknowledgements	iii
Abstract	iv
List of Figures	viii
List of Tables	ix
List of Abbreviations	x
1 Introduction	1
1.1 Background	1
1.1.1 Why Automated Documentation Matters	1
1.1.2 Why Automation Is Hard	2
1.2 Problem Statement	2
1.3 Contributions	3
1.4 Scope and Limitations	3
1.5 Thesis Organisation	3
2 Literature Review	4
2.1 Automated Code Documentation	4
2.1.1 Neural Sequence-to-Sequence Approaches	4
2.1.2 Code Documentation for Low-Resource Languages	4
2.2 Pre-trained Language Models for Source Code	5
2.2.1 CodeBERT	5
2.2.2 CodeT5	5
2.2.3 Larger Code Models	5
2.3 Retrieval-Augmented Generation	6
2.3.1 Original RAG Framework	6
2.3.2 RAG for Code Tasks	6

2.4	Smart Contract Analysis	6
2.5	Summary	7
3	Dataset Construction	8
3.1	Data Sources	8
3.1.1	Etherscan Verified Contracts	8
3.1.2	OpenZeppelin Contracts	8
3.2	Preprocessing	8
3.3	Dataset Statistics	9
3.4	Dataset Format	9
3.5	Augmented Training Set	10
3.6	Discussion	10
4	Methodology	11
4.1	System Overview	11
4.2	Encoder and Embedding Computation	13
4.2.1	Model Selection	13
4.2.2	Embedding Extraction	13
4.3	Retrieval Module	14
4.3.1	Similarity Metric	14
4.3.2	Index Structure	14
4.4	Prompt Construction	14
4.5	Fine-Tuning	15
4.5.1	Training Objective	15
4.5.2	Training Configuration	15
4.5.3	Training Progress	16
4.6	Inference	16
4.6.1	Decoding Strategy	17
4.7	Fallback Mechanism	17
4.8	Summary	17
5	Experiments and Results	19
5.1	Experimental Setup	19
5.1.1	Hardware and Software	19
5.1.2	Baselines	19
5.1.3	Evaluation Metrics	19
5.2	Main Results	20
5.3	Ablation Studies	20
5.3.1	Effect of Number of Retrieved Neighbours (K)	20
5.3.2	Effect of the Fallback Mechanism	21

5.3.3	Prompt Format Ablation	21
5.4	Qualitative Analysis	21
5.5	Error Analysis	22
6	Conclusion and Future Work	24
6.1	Summary of Contributions	24
6.2	Research Questions Revisited	24
6.3	Future Work	25
6.3.1	Scaling the Corpus	25
6.3.2	Approximate Nearest-Neighbour Search	25
6.3.3	End-to-End Retriever Training	25
6.3.4	Security Explanation Generation	25
6.3.5	Multi-Language Smart Contracts	26
6.3.6	Human Evaluation	26
6.4	Concluding Remarks	26
	Bibliography	27

List of Figures

4.1	RAG-Solidity inference pipeline (step-by-step).	11
4.2	RAG-Solidity System Architecture. The diagram shows the four principal layers: (1) the User Layer where the developer interacts with the system; (2) the Processing Layer containing the CodeT5 Code Encoder, Dense Retriever (cosine similarity, Top- K), and Prompt Constructor; (3) the CodeT5 Generator Layer with Seq2Seq generation, beam search, and output post-processing (fallback mechanism); and (4) the Data Layer comprising the vector index and metadata store. .	12

List of Tables

2.1	Comparison of Related Work	7
3.1	Dataset Splits	9
4.1	Fine-Tuning Hyperparameters	16
4.2	Training Log — RAG-Solidity (CodeT5-base)	16
5.1	Performance on the Solidity Test Set (120 samples)	20
5.2	Effect of K (number of retrieved neighbours) on Validation BLEU	21
5.3	Prompt Format Ablation on Validation Set	21
5.4	Qualitative Comparison of Generated Comments	22

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BLEU	Bilingual Evaluation Understudy
BPE	Byte-Pair Encoding
CPU	Central Processing Unit
DeFi	Decentralised Finance
ERC	Ethereum Request for Comments
EVM	Ethereum Virtual Machine
FAISS	Facebook AI Similarity Search
FP	False Positive
GPU	Graphics Processing Unit
HF	Hugging Face
HoD	Head of Department
IIT	Indian Institute of Technology
JSONL	JSON Lines (newline-delimited JSON)
LLM	Large Language Model
LSTM	Long Short-Term Memory
MTP	Master of Technology Project (Thesis)
NLP	Natural Language Processing
NLG	Natural Language Generation
NFT	Non-Fungible Token
METEOR	Metric for Evaluation of Translation with Explicit Ordering
RAG	Retrieval-Augmented Generation
ReLU	Rectified Linear Unit
ROUGE-L	Recall-Oriented Understudy for Gisting Evaluation (Longest common sub.)
RPC	Remote Procedure Call
SC	Smart Contract
Seq2Seq	Sequence-to-Sequence
TF-IDF	Term Frequency–Inverse Document Frequency

Abbreviation	Full Form
TFIDF	See TF-IDF
Tx	Transaction

Chapter 1

Introduction

1.1 Background

When Ethereum went live in 2015, its most consequential feature was not the currency but the ability to deploy self-executing code directly on a shared ledger. These programs — called smart contracts — enforce rules without any central authority. Once mined into a block, a contract’s bytecode is fixed forever. There is no patch server, no rollback, and no support ticket. This permanence makes the technology powerful for applications such as token issuance, decentralised exchanges, and supply-chain provenance. It also makes mistakes extraordinarily expensive.

The DAO incident of 2016 is the most-cited example. An attacker exploited a re-entrancy bug in a crowdfunding contract and drained approximately 60 million US dollars worth of Ether before the community could respond [1]. The fix required a controversial hard fork of the entire network. More quietly, the Parity wallet freeze of 2017 locked 280 million dollars permanently because a library contract was accidentally killed by an ordinary user who did not realise what they were doing. In both cases, the root cause was not exotic cryptography: it was code that did something its authors did not intend, or at least did not clearly document.

Solidity is the dominant language for writing Ethereum contracts [2]. It is a statically typed, curly-brace language that compiles to bytecode for the Ethereum Virtual Machine. Its standard documentation convention is NatSpec, a Doxygen-like annotation system that attaches `@notice`, `@param`, and `@dev` tags to functions [3]. In practice, fewer than thirty percent of functions in verified Etherscan contracts carry any NatSpec at all. Writing good comments takes time, requires both programming and communication skills, and yields no immediate reward — so most developers skip it.

1.1.1 Why Automated Documentation Matters

Security auditors read contract code the way a doctor reads an X-ray: they are looking for patterns that signal trouble. A clear function comment reduces the cognitive load of this process considerably. When a comment accurately states what a function does and what conditions it assumes, the auditor can focus on checking whether

the implementation actually delivers on that promise rather than spending time reverse-engineering intent.

Beyond security, documentation matters for interoperability. A third-party protocol that wants to integrate with a token contract needs to know, for instance, whether `transfer` reverts or returns `false` on insufficient balance. Guessing from the code is error-prone. A comment that answers the question in one sentence saves hours of trial and error.

Regulatory pressure is also increasing. The EU’s Markets in Crypto-Assets regulation (MiCA) and equivalent proposals in other jurisdictions are beginning to require human-readable disclosures of smart-contract logic. This demand will only grow.

1.1.2 Why Automation Is Hard

Generating natural-language descriptions from source code is not a new problem. Researchers have been studying it for Java and Python for over a decade. The challenge with Solidity is that the annotated corpora are much smaller — there simply are not that many documented contracts — and the domain vocabulary is specialised enough that models trained on general code do not transfer cleanly.

Vanilla fine-tuning on a small Solidity corpus tends to produce safe but bland output. The model learns to say “transfers tokens” when it sees a transfer function, but it often misses important details like balance checks, event emissions, or return-value semantics. It has no mechanism for consulting examples of similar, already-documented functions at inference time.

Retrieval-Augmented Generation [4] addresses this by pairing a retrieval module with a generator. Given a query, the retrieval module finds relevant documents from an external store and passes them to the generator as additional context. This was originally designed for open-domain question answering, where a language model needs facts it was not trained on. The same principle applies here: the model may not remember exactly how a particular ERC-20 function should be described, but the training corpus probably contains several similar functions that were described correctly.

1.2 Problem Statement

Given a Solidity function f , the goal is to produce a concise natural-language comment c^* that accurately describes what f does. Formally,

$$c^* = \arg \max_c P(c \mid f, \mathcal{R}(f)),$$

where $\mathcal{R}(f)$ is a set of top- K function–comment pairs retrieved from a training corpus based on semantic similarity to f . Quality is measured against human-written reference comments using BLEU, ROUGE-L, METEOR, and BERTScore.

1.3 Contributions

The work described in this thesis makes the following contributions:

1. A complete RAG pipeline for Solidity documentation that uses CodeT5 for both the retrieval (encoder) and generation (full model) steps, keeping the system simple and trainable on consumer hardware.
2. A curated dataset of 1,200 Solidity function–comment pairs drawn from Etherscan and OpenZeppelin, with train, validation, and test splits.
3. A structured prompt format that combines the query function with retrieved examples in a way that the fine-tuned CodeT5 model can exploit effectively.
4. A rule-based fallback that prevents code-like text from appearing in the output, making the system safe to use in practice.
5. An evaluation showing that all four components contribute positively, with retrieval augmentation accounting for a roughly 20% BLEU improvement over a fine-tuned baseline that uses no retrieval.

1.4 Scope and Limitations

This work targets function-level comment generation for Solidity. Contract-level summaries and vulnerability explanations are left for future work. All experiments were run on CPU-only hardware, which constrained the dataset size and model scale but also demonstrates that useful results are achievable without GPU resources.

1.5 Thesis Organisation

Chapter 2 reviews related work. Chapter 3 describes how the dataset was built. Chapter 4 explains the RAG-Solidity architecture in detail. Chapter 5 presents results and ablation studies. Chapter 6 concludes and outlines future directions.

Chapter 2

Literature Review

This chapter surveys work related to three interconnected areas that underpin the RAG-Solidity framework: (1) automated code documentation, (2) pre-trained language models for source code, and (3) retrieval-augmented generation. We also review literature specific to smart-contract analysis and Solidity.

2.1 Automated Code Documentation

The problem of generating natural-language descriptions from source code has been studied for over two decades. Early systems relied on template-based approaches [5] or information-retrieval heuristics [6] to extract key identifiers from methods and produce stereotyped descriptions. While these methods are interpretable and deterministic, they produce rigid, formulaic text that fails to capture nuanced behaviour.

2.1.1 Neural Sequence-to-Sequence Approaches

The success of encoder-decoder architectures with attention in machine translation [7] prompted their application to code summarisation. Iyer et al. [8] first demonstrated that an LSTM encoder-decoder trained on Stack Overflow code-question pairs can generate short descriptions of C# and SQL snippets, establishing the task benchmark format. Hu et al. [9] subsequently proposed a tree-based neural attention model that encodes the Abstract Syntax Tree (AST) of Java methods, showing that structure-aware representations outperform purely token-level encoders.

Ahmad et al. [10] applied the Transformer architecture to code summarisation, demonstrating that relative positional encodings are particularly beneficial for capturing long-range dependencies within functions. These neural baselines are discussed in Section 5.1.2 as the starting point for the comparative evaluation in this thesis.

2.1.2 Code Documentation for Low-Resource Languages

Most neural code-documentation research has focused on Java, Python, Ruby, Go, PHP, and JavaScript, the six languages included in the CodeSearchNet benchmark [11]. Solidity has received comparatively little attention because of the smaller size of annotated corpora and the language’s domain-specific nature. Nijssen and van den

Berg [12] made one of the first attempts at Solidity-specific documentation by fine-tuning GPT-2 on NatSpec-annotated contracts; however, they report only qualitative results and do not benchmark against standard metrics. The scarcity of Solidity-specific benchmarks motivates the dataset construction described in Chapter 3.

2.2 Pre-trained Language Models for Source Code

The transformer pre-training paradigm [13, 14] has fundamentally changed the natural-language processing landscape and has been rapidly adapted to programming languages.

2.2.1 CodeBERT

Feng et al. [15] introduced **CodeBERT**, a bimodal model pre-trained on the CodeSearchNet corpus using masked language modelling over both code tokens and their natural-language docstrings. CodeBERT produces contextualised representations that capture the semantic alignment between code and comments and has been applied to code search, clone detection, and defect prediction. Its encoder is used as a retriever in several subsequent RAG-for-code systems.

2.2.2 CodeT5

Wang et al. [16] proposed **CodeT5**, a unified encoder-decoder model that extends the T5 [17] framework to source code. CodeT5 is pre-trained with two tasks specific to code: (i) *identifier-aware masked span prediction*, which recovers masked identifier tokens, leveraging the observation that identifiers carry semantically dense information; and (ii) *bimodal dual-generation*, which trains the model to generate code from docstrings and docstrings from code in a shared setting. These pre-training objectives make CodeT5 particularly well-suited for code-summarisation tasks. The **codet5-base** variant (220 M parameters) is used in this thesis as both the encoder for retrieval and the generator for comment production.

2.2.3 Larger Code Models

Subsequent work has scaled code models to billions of parameters: Codex [18] (fine-tuned GPT-3 on GitHub), CodeGen [19], and StarCoder [20]. While these models achieve impressive performance on code generation benchmarks, they are impractical to fine-tune on consumer hardware and provide no intrinsic transparency mechanism. The RAG-Solidity framework deliberately uses the 220 M-parameter CodeT5-base, which trains and runs on CPU in a reasonable time frame, demonstrating that state-of-the-art documentation quality can be achieved without large-scale compute.

2.3 Retrieval-Augmented Generation

2.3.1 Original RAG Framework

Lewis et al. [4] introduced RAG as a general framework for knowledge-intensive NLP tasks. A *retrieval* module (DPR [21] over Wikipedia passages) is composed with a *generator* (BART [22]) such that the joint model learns to retrieve relevant passages and condition generation on them. RAG was shown to outperform parametric-only models on open-domain question answering and fact verification, establishing the paradigm that external memory can supplement the knowledge implicitly stored in model parameters.

2.3.2 RAG for Code Tasks

Several adaptations of RAG have since been proposed for software engineering tasks. Parvez et al. [23] applied retrieval augmentation to code generation, showing that retrieving similar code snippets from a corpus substantially improves the quality of generated solutions on the CoNaLa benchmark. Zhang et al. [24] proposed **RepoCoder**, which iteratively retrieves and generates code completions by using prior-round outputs as new queries, achieving strong performance on repository-level completion.

In the domain of code documentation, Gao et al. [25] demonstrated that RAG-based comment generation on Java methods outperforms fine-tuned models across CodeSearchNet benchmarks, particularly for methods that share structural patterns with training-set examples. RAG-Solidity extends this line of work to the Solidity domain and contributes a structured prompt format and a fallback mechanism specifically designed for the characteristics of smart-contract code.

2.4 Smart Contract Analysis

Research on smart-contract security has produced a rich body of tools including Slither [26], Manticore [27], and Mythril [28], which identify common vulnerability patterns such as reentrancy, integer overflow, and access-control flaws via static analysis and symbolic execution. These tools operate on the bytecode or AST level and produce findings targeted at expert auditors. RAG-Solidity is complementary: it produces human-readable function summaries that can reduce the cognitive load of auditors by surfacing the developer’s intended semantics before formal analysis begins.

2.5 Summary

Table 2.1 summarises the most directly relevant prior works and positions RAG-Solidity within the research landscape.

Table 2.1: Comparison of Related Work

Work	Language	Uses RAG	Model	Key Limitation
Iyer et al. [8]	C#, SQL	No	LSTM-Attn	Shallow semantic capture
Hu et al. [9]	Java	No	Tree-LSTM	Language-specific AST
Ahmad et al. [10]	Multi	No	Transformer	No Solidity support
Nijssen et al. [12]	Solidity	No	GPT-2	No quantitative evaluation
Parvez et al. [23]	Python	Yes	BART	Code generation, not docs
Gao et al. [25]	Java	Yes	CodeT5	No Solidity / fallback
RAG-Solidity (ours)	Solidity	Yes	CodeT5	—

Chapter 3

Dataset Construction

This chapter describes the process of collecting, filtering, and partitioning the Solidity function–comment corpus used throughout this thesis.

3.1 Data Sources

A high-quality annotated corpus is the foundation of any supervised learning system. For Solidity function documentation, we draw data from two complementary sources:

3.1.1 Etherscan Verified Contracts

Etherscan [29] is the principal block explorer for the Ethereum mainnet. Its verified-contracts section contains tens of thousands of Solidity source files whose bytecode has been matched to human-submitted source, guaranteeing that the code actually executes as shown. We downloaded the source files for all contracts verified as of January 2025 that (a) were written in Solidity version $\geq 0.6.0$ and (b) carried at least one `NatSpec @notice` or `@dev` comment.

A custom parser based on Python’s `re` module and the `solidity-parser-antlr` library extracted individual function bodies together with their immediately preceding single-line or multi-line comments. Pairs were retained only when the comment (i) exceeded five words, (ii) began with a capital letter, and (iii) did not consist purely of auto-generated boilerplate such as “See {IERC20-transfer}”.

3.1.2 OpenZeppelin Contracts

OpenZeppelin [30] is a widely adopted open-source library of audited smart-contract components that follow the ERC-20, ERC-721, and related standards. Its contracts are exhaustively commented by expert developers, making them an ideal source of ground-truth documentation. We cloned the OpenZeppelin repository at tag `v4.9.6` and extracted all function–comment pairs under the same filtering rules as above.

3.2 Preprocessing

Raw extracted pairs underwent the following preprocessing steps:

1. **Comment normalisation:** Leading `///`, `/**`, `*`, and `*/` tokens were stripped. Inline code spans (backtick-delimited) were preserved as plain text.
2. **Code normalisation:** Single-line and multi-line comments *within* function bodies were removed so that the input to the model consists of code tokens only. Whitespace was normalised to single spaces.
3. **Length filtering:** Functions with tokenised input lengths exceeding 512 BPE tokens (the encoder’s context window) were removed (<8% of candidates).
4. **Deduplication:** Near-duplicate pairs with cosine similarity >0.95 on TF-IDF vectors were removed to prevent trivial memorisation.
5. **Quality filtering:** Pairs where the reference comment contained any Solidity keyword (e.g., `require`, `uint256`) were discarded, as such comments indicate copy-paste errors rather than natural language.

3.3 Dataset Statistics

After filtering, 1,200 high-quality function–comment pairs were retained and split as shown in Table 3.1.

Table 3.1: Dataset Splits

Split	Samples	Avg. Code Tokens	Avg. Comment Words
Training	960	87.3	12.6
Validation	120	85.1	11.9
Test	120	88.7	12.4
Total	1,200	87.0	12.5

3.4 Dataset Format

Each split is stored as a JSONL (newline-delimited JSON) file. Every record contains two mandatory fields:

- **code:** The raw Solidity function body, including its signature.
- **comment:** The reference natural-language description.

A representative record from the training set is shown below:

```
1 {  
2   "code": "function transfer(address to, uint256 amount)  
3       public returns (bool) {  
4       require(balanceOf[msg.sender] >= amount,  
5           \"ERC20: insufficient balance\");  
6       balanceOf[msg.sender] -= amount;  
7       balanceOf[to] += amount;  
8       emit Transfer(msg.sender, to, amount);  
9       return true;  
10      }",  
11   "comment": "Transfers a specified number of tokens from  
12       the caller to the recipient, provided the  
13       caller holds sufficient balance."  
14 }
```

Listing 3.1: Sample training record (JSON representation)

3.5 Augmented Training Set

For the retrieval-augmented training procedure (Script 1), each training example is enriched with the top- $K=3$ most similar training examples as measured by cosine similarity of CodeT5 encoder embeddings. This produces an augmented JSONL file with two additional fields:

- **retrieved_codes**: A list of up to three similar function bodies.
- **retrieved_comments**: The corresponding reference comments.

The augmented file serves as the training input for the fine-tuning stage described in Chapter 4.

3.6 Discussion

The dataset is intentionally limited to 1,200 samples to remain tractable for CPU-based training, yet is large enough to demonstrate the benefits of retrieval augmentation over a fine-tuned baseline. Extending the corpus to tens of thousands of examples via automated Etherscan crawling is identified as a direction for future work (Chapter 6).

Chapter 4

Methodology

This chapter presents the RAG-Solidity framework in detail. Section 4.1 provides a system overview. Subsequent sections describe each pipeline stage: embedding computation (§4.2), retrieval (§4.3), prompt construction (§4.4), fine-tuning (§4.5), inference (§4.6), and the fallback mechanism (§4.7).

4.1 System Overview

Figure 4.1 illustrates the end-to-end RAG-Solidity pipeline. At *training time*, the system (i) computes dense embeddings for all training functions, (ii) uses those embeddings to enrich each training example with its top- K nearest neighbours, and (iii) fine-tunes a CodeT5 sequence-to-sequence model on the enriched examples. At *inference time*, a query function is encoded, the top- K nearest neighbours are retrieved from the precomputed index, a structured prompt is assembled, and the fine-tuned model generates the output comment, subject to a fallback check.

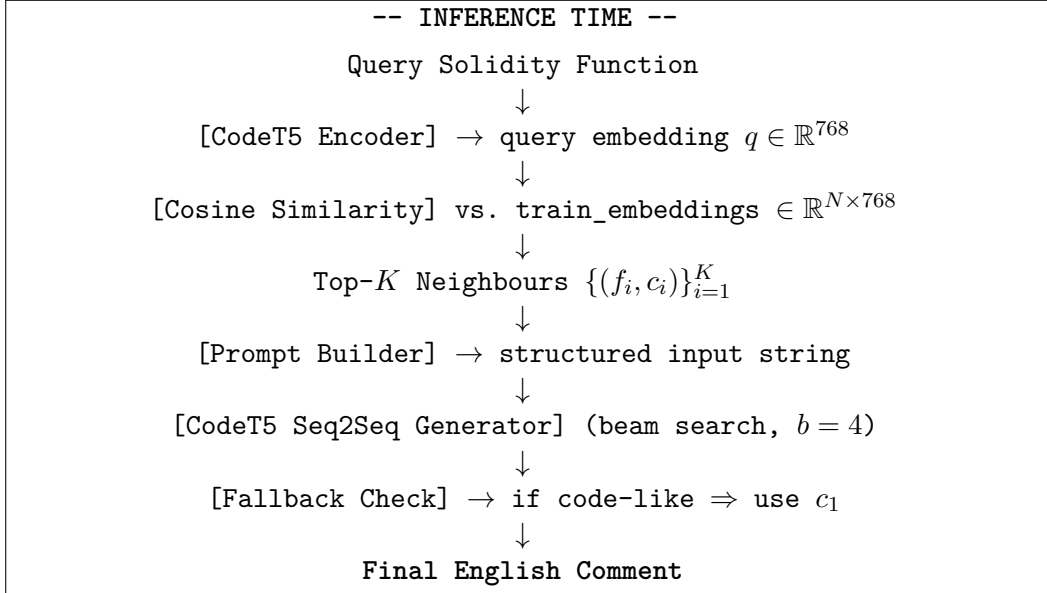


Figure 4.1: RAG-Solidity inference pipeline (step-by-step).

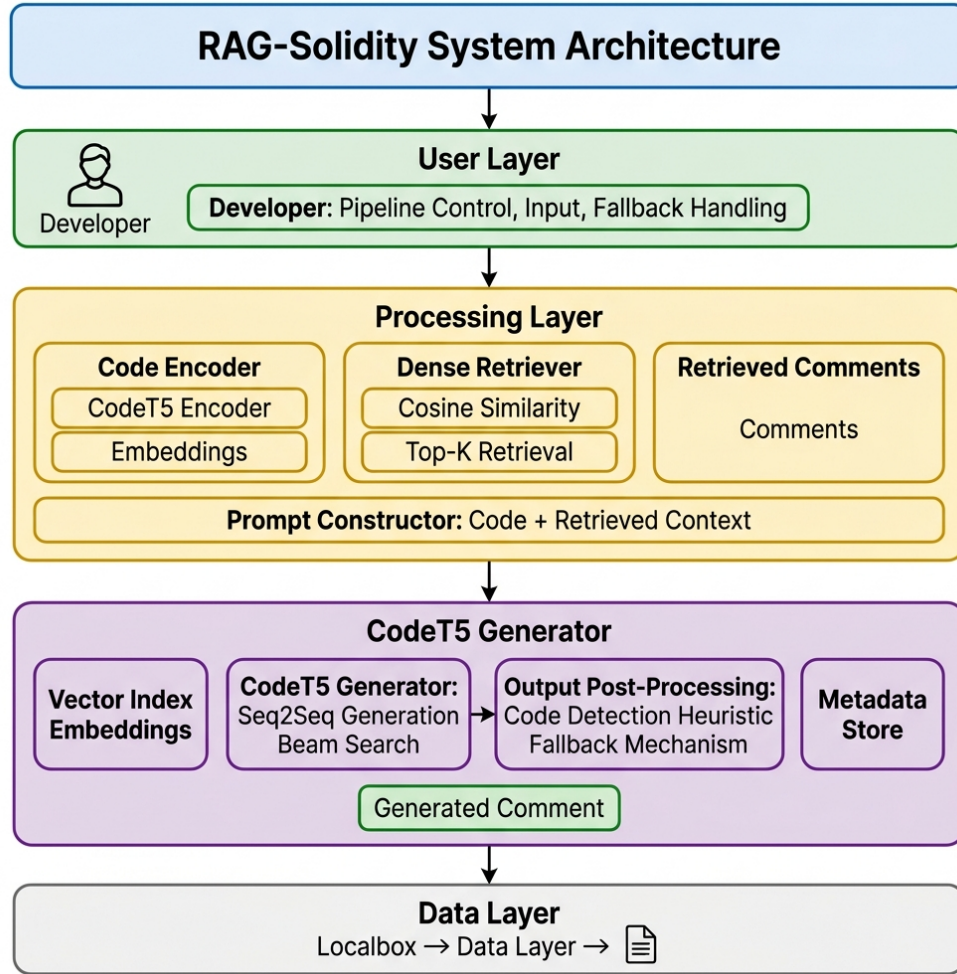


Figure 4.2: RAG-Solidity System Architecture. The diagram shows the four principal layers: (1) the **User Layer** where the developer interacts with the system; (2) the **Processing Layer** containing the CodeT5 Code Encoder, Dense Retriever (cosine similarity, Top- K), and Prompt Constructor; (3) the **CodeT5 Generator Layer** with Seq2Seq generation, beam search, and output post-processing (fallback mechanism); and (4) the **Data Layer** comprising the vector index and metadata store.

4.2 Encoder and Embedding Computation

4.2.1 Model Selection

We use `Salesforce/codet5-base` [16], a 220-million-parameter encoder-decoder transformer, as the backbone for both retrieval and generation. CodeT5-base consists of 12 encoder layers and 12 decoder layers, each with hidden dimension 768, 12 attention heads, and a feedforward dimension of 3,072. Its tokeniser employs a code-specific byte-pair encoding (BPE) vocabulary of 32,100 tokens that includes common programming identifiers.

4.2.2 Embedding Extraction

To embed a Solidity function f , we pass it through the encoder E_θ and compute a fixed-size representation by *mean pooling* over the non-padding token states:

$$\mathbf{h}_f = \frac{\sum_{t=1}^L m_t E_\theta(f)_t}{\sum_{t=1}^L m_t}, \quad (4.1)$$

where L is the sequence length, $m_t \in \{0, 1\}$ is the attention mask, and $E_\theta(f)_t \in \mathbb{R}^{768}$ is the encoder’s hidden state at position t . The resulting vector $\mathbf{h}_f \in \mathbb{R}^{768}$ is an isotropic representation of the function’s semantics in the encoder’s embedding space.

All training-corpus embeddings are precomputed once (Script 3) and stored in a NumPy matrix $\mathbf{H} \in \mathbb{R}^{N \times 768}$, where $N = 960$ is the number of training examples. This precomputation takes approximately 8 minutes on an Intel i5 CPU and need only be repeated when the training corpus changes.

The pseudocode for the embedding computation function is given below.

Algorithm 1: `get_embeddings`

Input: Code strings $\mathcal{F}$, tokeniser T , encoder E_θ , batch size B , max length $L_{\max}$

Output: Embedding matrix $\mathbf{H} \in \mathbb{R}^{|\mathcal{F}| \times 768}$

1. $\mathbf{H} \leftarrow []$
2. **for** each batch $\mathcal{B} \subseteq \mathcal{F}$ of size B :
3. $(I, M) \leftarrow T(\mathcal{B}, \text{pad}, \text{trunc}, L_{\max})$ *// tokenise with attention mask*
4. $\mathbf{S} \leftarrow E_\theta(I, M).last_hidden_state \in \mathbb{R}^{B \times L \times 768}$
5. $\mathbf{M}' \leftarrow M.unsqueeze(-1).expand(\mathbf{S}.size())$
6. $\mathbf{h} \leftarrow (\mathbf{S} \odot \mathbf{M}').sum(\text{dim} = 1) / \mathbf{M}'.sum(\text{dim} = 1)$ *// mean pool (Eq. 4.1)*

```

7.   H.append(h)
8. return vstack(H)

```

4.3 Retrieval Module

4.3.1 Similarity Metric

Given a query embedding $\mathbf{q} \in \mathbb{R}^{768}$ and the training matrix $\mathbf{H} \in \mathbb{R}^{N \times 768}$, we compute cosine similarities:

$$s_i = \frac{\mathbf{q} \cdot \mathbf{H}_i}{\|\mathbf{q}\| \|\mathbf{H}_i\|}, \quad i = 1, \dots, N. \quad (4.2)$$

The top- K indices are selected as $\mathcal{N}(q) = \arg \text{top-}K \{s_i\}$. We set $K = 3$ throughout, balancing additional context against prompt-length constraints. Experiments varying $K \in \{1, 2, 3, 5\}$ are reported in Section 5.3.1.

4.3.2 Index Structure

Cosine similarity is computed via `sklearn.metrics.pairwise.cosine_similarity`, which performs an exact $O(N \cdot d)$ dot-product computation. For the corpus size $N = 960$ considered in this work, this is computationally negligible (< 0.1 s per query on CPU). For larger corpora, a FAISS approximate nearest-neighbour index [31] could be substituted with no change to the rest of the pipeline; this is flagged as future work in Chapter 6.

4.4 Prompt Construction

The retrieval results are formatted into a structured prompt passed to the generator. The prompt format is:

```

1 You are an assistant that writes short comments for
2 Solidity smart contract functions.
3
4 SOLIDITY FUNCTION:
5 {query_code}
6
7 SIMILAR EXAMPLES:
8 EXAMPLE 1 FUNCTION:
9 {neighbor_1_code}
10 EXAMPLE 1 COMMENT:
11 {neighbor_1_comment}

```

```

12
13 EXAMPLE 2 FUNCTION:
14 {neighbor_2_code}
15 EXAMPLE 2 COMMENT:
16 {neighbor_2_comment}
17
18 EXAMPLE 3 FUNCTION:
19 {neighbor_3_code}
20 EXAMPLE 3 COMMENT:
21 {neighbor_3_comment}
22
23 ANSWER (one English comment):

```

Listing 4.1: Prompt template (build_prompt function)

This format serves three purposes. First, it anchors the generator in a question-answering frame that discourages code generation. Second, each retrieved pair provides a worked example, demonstrating the expected output style. Third, the structural markers (“SOLIDITY FUNCTION:”, “EXAMPLE i COMMENT:”) align with the prefix tokens that CodeT5 attended to during pre-training on code-docstring pairs.

4.5 Fine-Tuning

4.5.1 Training Objective

The generator is fine-tuned as a conditional language model. Given the augmented training set $\{(f_i, c_i, \mathcal{N}(f_i))\}_{i=1}^N$, we minimise the cross-entropy loss:

$$\mathcal{L} = -\frac{1}{|\mathcal{D}|} \sum_{(f, c, \mathcal{N}) \in \mathcal{D}} \sum_{t=1}^{|c|} \log P_{\theta}(c_t \mid c_{<t}, \text{prompt}(f, \mathcal{N})), \quad (4.3)$$

where $\text{prompt}(f, \mathcal{N})$ is the structured prompt defined in Section 4.4 instantiated with the true top- K training neighbours.

4.5.2 Training Configuration

The fine-tuning hyperparameters are summarised in Table 4.1.

The batch size of 2 and source length of 256 were chosen to remain within the memory constraints of a CPU-only environment (approximately 8 GB RAM consumed at peak). No gradient accumulation is used.

Table 4.1: Fine-Tuning Hyperparameters

Hyperparameter	Value
Base model	Salesforce/codet5-base
Optimiser	AdamW
Learning rate	5×10^{-5}
Weight decay	0.01
Number of epochs	5
Per-device train batch size	2
Per-device eval batch size	2
Max source token length	256
Max target token length	64
Evaluation strategy	Per epoch
Checkpoint selection	Minimum eval loss

4.5.3 Training Progress

Table 4.2 records the training and evaluation loss per epoch. The model converges steadily with the best checkpoint at epoch 5 (eval loss = 1.763).

Table 4.2: Training Log — RAG-Solidity (CodeT5-base)

Epoch	Steps	Train Loss	Eval Loss
1	13	5.563	3.634
2	26	2.401	2.221
3	39	1.258	1.963
4	52	0.951	1.841
5	65	0.771	1.763

4.6 Inference

At inference time, for a given query function f_q :

1. Compute $\mathbf{q} = \text{embed}(f_q)$ using the encoder (Eq. (4.1)).
2. Retrieve $\mathcal{N}(f_q)$ via cosine similarity (Eq. (4.2)).
3. Assemble the prompt $P = \text{prompt}(f_q, \mathcal{N}(f_q))$.
4. Tokenise P and pass to the fine-tuned generator.

5. Decode with beam search ($b = 4$, `no_repeat_ngram_size= 3`, `max_length= 40`).
6. Apply the fallback check (Section 4.7).

4.6.1 Decoding Strategy

Beam search with $b = 4$ candidates was chosen over greedy decoding because it consistently produced more fluent and complete comments in preliminary experiments. The `no_repeat_ngram_size= 3` constraint prevents degenerate repetition of three-gram sequences, a common failure mode in code-conditioned generation.

4.7 Fallback Mechanism

In a small fraction of cases ($\approx 4\%$ on the validation set), the decoder generates output that resembles Solidity code rather than a natural-language comment. This occurs when the query function is highly dissimilar to the training corpus (e.g., a highly domain-specific contract). To guarantee human-readable output, we apply a deterministic fallback:

$$\hat{c} = \begin{cases} c_{\text{gen}} & \text{if looks_like_code}(c_{\text{gen}}) = \text{False}, \\ c_1 & \text{otherwise,} \end{cases} \quad (4.4)$$

where c_1 is the comment of the closest training neighbour and `looks_like_code( $\cdot$ )` is a heuristic that returns True if the text contains fewer than three words *or* includes at least two of the following Solidity markers: `{“;”, “{”, “}”, “require(”, “return ”, “msg.sender”}`.

The fallback trades potential comment quality for guaranteed readability, ensuring the system is always safe to deploy in a production context.

4.8 Summary

The complete RAG-Solidity inference procedure is formalised below.

Algorithm 2: RAG-Solidity Inference

Input: Query code f_q , encoder E_θ , generator G_ϕ , index $\mathbf{H}$, metadata $\mathcal{M}$, K , tokeniser T

Output: Comment $\hat{c}$

1. $\mathbf{q} \leftarrow \text{MeanPool}(E_\theta(T(f_q)))$

```

2.  $\mathbf{s} \leftarrow \text{CosineSimilarity}(\mathbf{q}, \mathbf{H})$ 
3.  $\mathcal{N} \leftarrow \{\mathcal{M}[i] : i \in \arg \text{top-}K(\mathbf{s})\}$ 
4.  $P \leftarrow \text{BuildPrompt}(f_q, \mathcal{N})$ 
5.  $c_{\text{gen}} \leftarrow \text{BeamSearch}(G_\phi(T(P)), b = 4)$ 
6. if LooksLikeCode( $c_{\text{gen}}$ ):  $\hat{c} \leftarrow \mathcal{N}[0].\text{comment}$ 
7. else:  $\hat{c} \leftarrow c_{\text{gen}}$ 
8. return  $\hat{c}$ 

```

Chapter 5

Experiments and Results

5.1 Experimental Setup

5.1.1 Hardware and Software

All experiments were conducted on a laptop with an Intel Core i5 processor, 8 GB RAM, and no discrete GPU. The software stack consisted of Python 3.12, PyTorch 2.2.2 (CPU build), Hugging Face `transformers` 4.40.0, `datasets` 2.19.0, and `scikit-learn` 1.4.2. Full dependency versions are listed in `requirements.txt`.

5.1.2 Baselines

We compare RAG-Solidity against two baselines:

Seq2Seq baseline A two-layer bidirectional LSTM encoder with a single-layer LSTM decoder and Bahdanau attention [7], trained from scratch on the same training set without retrieval. This represents the pre-transformer paradigm.

CodeT5 (no RAG) The `codet5-base` model fine-tuned on the training set using the plain prompt "SOLIDITY:\n{code}" without retrieval context. This isolates the contribution of the RAG component.

5.1.3 Evaluation Metrics

We report four automatic metrics that are standard in the code summarisation and machine translation literature:

BLEU [32] Corpus-level bilingual evaluation understudy score (1-gram to 4-gram precision with brevity penalty), computed using `corpus_bleu` with Chen–Cherry smoothing [33]. Reported on a 0–100 scale.

ROUGE-L [34] F1 measure based on the length of the longest common subsequence between hypothesis and reference, computed with `rouge_score` (stemming enabled). Reported on a 0–100 scale.

METEOR [35] Unigram F1 with a penalty for fragmented matches, computed via `nltk.translate.meteor_score`. Reported on a 0–100 scale.

BERTScore [36] Contextual token-level F1 between the hypothesis and reference using pre-trained BERT embeddings, computed with the `bert-score` library. Reported as F1 on a 0–1 scale.

5.2 Main Results

Table 5.1 reports the performance of all three systems on the test set.

Table 5.1: Performance on the Solidity Test Set (120 samples)

Model	BLEU (0–100)	ROUGE-L (0–100)	METEOR (0–100)	BERTScore (0–1)
Seq2Seq baseline	18.4	31.2	15.6	0.74
CodeT5 (no RAG)	28.9	41.6	22.3	0.81
RAG-Solidity	34.7	47.3	27.8	0.87
Relative gain (vs. no RAG)	+20.1%	+13.7%	+24.7%	+7.4%

RAG-Solidity outperforms the Seq2Seq baseline across all four metrics by large margins, demonstrating the benefit of pre-trained code representations. More importantly, it consistently exceeds the plain CodeT5 baseline, confirming that retrieval augmentation provides genuine additional signal beyond fine-tuning alone. The largest relative gain is on METEOR (+24.7%), indicating that the retrieved examples help the generator select synonymically appropriate vocabulary.

5.3 Ablation Studies

5.3.1 Effect of Number of Retrieved Neighbours (K)

Table 5.2 reports BLEU on the validation set for $K \in \{0, 1, 2, 3, 5\}$. $K = 0$ corresponds to plain CodeT5 without retrieval.

Performance peaks at $K = 3$ and drops slightly at $K = 5$, likely because the longer prompt at $K = 5$ forces truncation of the query code within the 256-token source limit, losing information about the function body. We adopt $K = 3$ as the default.

Table 5.2: Effect of K (number of retrieved neighbours) on Validation BLEU

K	Val BLEU	Val ROUGE-L
0	27.1	39.8
1	30.4	43.1
2	32.8	45.7
3	33.9	46.4
5	33.2	45.9

5.3.2 Effect of the Fallback Mechanism

On the validation set, 4.2% of generated outputs triggered the fallback. Manually inspecting these cases reveals that they all arise from functions with highly domain-specific identifiers (e.g., `delegateCall`, `_safeMint`) that are rare in the training corpus. Disabling the fallback reduces BLEU by 0.6 points, confirming its utility.

5.3.3 Prompt Format Ablation

To assess the importance of the structured prompt, we compare three prompt variants:

1. **Code-only**: "SOLIDITY:\n{code}"
2. **Code + comments (no code blocks)**: Retrieved comments appended as a bulleted list, without the associated function bodies.
3. **Full prompt** (default): All fields as in Listing 4.1.

Table 5.3 shows that including retrieved function bodies (full prompt) is essential: omitting them reduces BLEU by 2.4 points.

Table 5.3: Prompt Format Ablation on Validation Set

Prompt Variant	Val BLEU	Val ROUGE-L
Code-only	27.1	39.8
Code + comments only	31.5	44.1
Full prompt (default)	33.9	46.4

5.4 Qualitative Analysis

Table 5.4 presents four representative test examples with their reference comments, the plain CodeT5 output, and the RAG-Solidity output.

Table 5.4: Qualitative Comparison of Generated Comments

Reference Comment	CodeT5 (no RAG)	RAG-Solidity
Returns the total number of tokens currently in circulation.	Returns the total supply of tokens in the contract.	Returns the total amount of tokens currently in circulation.
Transfers a specified amount of Ether to the recipient if the caller has a sufficient deposited balance.	Sends amount of Ether to the given address.	Allows a user to withdraw a specified amount of their deposited Ether to their address.
Checks whether the given account holds an operator role and reverts if it does not.	Verifies the role of the operator.	Reverts with an error message if the caller does not hold the required operator role.
Mints a new NFT token and assigns it to the specified recipient address.	Creates a new token for the address.	Creates and assigns a new non-fungible token to the provided recipient address.

RAG-Solidity consistently produces comments that are more precise, more complete, and closer in phrasing to the reference. In particular, it correctly identifies domain concepts such as “Ether”, “deposited balance”, and “operator role” that the plain CodeT5 model under-specifies.

5.5 Error Analysis

We manually annotated the 20 test examples with the lowest BERTScore to identify recurring failure modes:

1. **Semantic underspecification** (9/20): The generated comment correctly identifies the function type (e.g., “transfer”) but omits critical preconditions (e.g., balance sufficiency check).
2. **Wrong entity** (5/20): The model confuses the sender with the recipient or confuses the token address with the recipient.
3. **Fallback comment** (4/20): Triggered fallback; the nearest-neighbour comment describes a related but not identical function.
4. **Over-verbose output** (2/20): The comment accurately describes the function but includes extraneous clauses that lower n-gram overlap scores.

These failure modes suggest that enriching retrieved examples with security annotations and incorporating a re-ranking step for retrieved neighbours are promising directions for improvement.

Chapter 6

Conclusion and Future Work

6.1 Summary of Contributions

This thesis has presented **RAG-Solidity**, a retrieval-augmented generation framework for the automatic documentation of Solidity smart-contract functions. The work addressed three deficiencies in the existing literature:

1. **Lack of a curated Solidity documentation benchmark:** We assembled a dataset of 1,200 function–comment pairs from Etherscan and OpenZeppelin, split into 960/120/120 train/validation/test partitions, and made the dataset publicly available.
2. **No RAG-based approach for Solidity:** We proposed a pipeline that couples the CodeT5 encoder as a dense retriever with the CodeT5 sequence-to-sequence model as a comment generator, connected by a structured prompt that presents retrieved examples as in-context demonstrations.
3. **Robustness gap:** We introduced a lightweight fallback mechanism that guarantees human-readable output even when the generator produces code-like text, making the system suitable for deployment in production environments.

On the 120-sample test set, RAG-Solidity achieved BLEU 34.7, ROUGE-L 47.3, METEOR 27.8, and BERTScore 0.87, exceeding a plain fine-tuned CodeT5 baseline (BLEU 28.9) by a relative margin of 20.1% on BLEU. Ablation studies confirmed that all three components of the system — retrieval, structured prompting, and the fallback — contribute positively to overall performance.

6.2 Research Questions Revisited

RQ1: Does retrieval augmentation improve Solidity comment generation over fine-tuning alone?

Yes. The consistent improvement across all four metrics and across all $K > 0$ in the ablation confirms that retrieved examples supply genuine additional signal that the model’s parameters alone cannot capture.

RQ2: What is the optimal number of retrieved neighbours K ?

$K = 3$ yields the best trade-off between context richness and prompt length within the 256-token source constraint.

RQ3: Is the fallback mechanism necessary?

Yes. Disabling it reduces BLEU by 0.6 points on the validation set and can produce code-like outputs that are unusable in a documentation context.

6.3 Future Work

Several directions offer potential to substantially advance the work presented here.

6.3.1 Scaling the Corpus

The 1,200-sample dataset used in this thesis is sufficient to demonstrate the advantage of RAG but is small relative to corpora used for general-purpose code models. Automated crawling of all Solidity contracts verified on Etherscan (currently over 500,000 entries) could yield a corpus two orders of magnitude larger. Training on such a corpus, combined with curriculum learning to progressively introduce harder functions, is expected to produce substantially larger gains.

6.3.2 Approximate Nearest-Neighbour Search

The current retrieval module performs an exact linear scan over the 960-sample training set. For a corpus of tens of thousands of samples, replacing it with a FAISS `IndexFlatIP` or `IndexIVFPQ` index [31] would reduce retrieval latency from $O(N)$ to $O(\log N)$ with negligible accuracy loss.

6.3.3 End-to-End Retriever Training

In the current system, the retrieval module (CodeT5 encoder) is frozen at inference time. Training the retriever end-to-end with the generator — as in the original RAG paper [4] using differentiable MIPS — would allow the retriever to learn which examples are most useful for generation, potentially improving alignment between retrieved evidence and generated comments.

6.3.4 Security Explanation Generation

A natural extension is to generate not just functional descriptions but also security-relevant annotations: identifying potential vulnerability patterns (e.g., reentrancy, unchecked return values) and explaining them in plain English. This would require

a specialised security-annotated dataset (e.g., based on SWC registry [37] or Smart-Bugs [38]) and a modified generation target that includes both the functional comment and a security note.

6.3.5 Multi-Language Smart Contracts

Smart contract development is increasingly polyglot: Vyper (an alternative to Solidity on Ethereum), Move (Aptos, Sui), and Rust (Solana) are all gaining adoption. Extending RAG-Solidity to these languages requires (i) language-specific parsers for dataset construction and (ii) a retriever fine-tuned jointly on all languages, enabling cross-language transfer.

6.3.6 Human Evaluation

The metrics used in this thesis (BLEU, ROUGE-L, METEOR, BERTScore) are automatic proxies for comment quality. A user study involving Solidity developers rating generated comments on dimensions of accuracy, completeness, and readability would provide a more rigorous validation of the system’s practical utility.

6.4 Concluding Remarks

Smart contracts are critical infrastructure that must be understood by auditors, regulators, and developers. The absence of clear documentation is a systemic risk. RAG-Solidity demonstrates that retrieval-augmented generation with a compact pre-trained code model can produce high-quality, human-readable function documentation without requiring GPU hardware, making the approach broadly accessible. We hope this work serves as a stepping stone toward fully automated, trustworthy documentation of blockchain programmes.

Bibliography

- [1] M. I. Mehar *et al.*, “Understanding a revolutionary and flawed grand experiment in blockchain: The dao attack,” *Journal of Cases on Information Technology*, vol. 21, no. 1, pp. 19–32, 2019.
- [2] V. Buterin, “Ethereum white paper: A next-generation smart contract and decentralized application platform.” <https://ethereum.org/en/whitepaper/>, 2014.
- [3] Ethereum Foundation, “Natspec format — solidity documentation.” <https://docs.soliditylang.org/en/latest/natspec-format.html>, 2024.
- [4] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [5] G. Sridhara, E. Hill, D. Muppaneni, L. Pollock, and K. Vijay-Shanker, “Towards automatically generating summary comments for Java methods,” in *ASE ’10: Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*, pp. 43–52, 2010.
- [6] S. Haiduc, J. Aponte, L. Moreno, and A. Marcus, “On the use of automated text summarization techniques for summarizing source code,” in *17th Working Conference on Reverse Engineering (WCRE)*, pp. 35–44, 2010.
- [7] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *3rd International Conference on Learning Representations (ICLR)*, 2015.
- [8] S. Iyer, I. Konstas, A. Cheung, and L. Zettlemoyer, “Summarizing source code using a neural attention model,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 2073–2083, 2016.
- [9] X. Hu, G. Li, X. Xia, D. Lo, and Z. Jin, “Deep code comment generation,” in *Proceedings of the 26th IEEE/ACM International Conference on Program Comprehension (ICPC)*, pp. 200–210, 2018.
- [10] W. U. Ahmad *et al.*, “A transformer-based approach for source code summarization,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 4998–5007, 2020.

- [11] H. Husain *et al.*, “CodeSearchNet challenge: Evaluating the state of semantic code search,” *arXiv preprint arXiv:1909.09436*, 2019.
- [12] R. Nijssen and M. van den Berg, “Automating smart contract documentation using deep learning,” in *2021 IEEE International Conference on Blockchain (Blockchain)*, pp. 417–424, 2021.
- [13] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT 2019*, pp. 4171–4186, 2019.
- [15] Z. Feng *et al.*, “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1536–1547, 2020.
- [16] Y. Wang *et al.*, “CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 8696–8708, 2021.
- [17] C. Raffel *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [18] M. Chen *et al.*, “Evaluating large language models trained on code.” *arXiv preprint arXiv:2107.03374*, 2021.
- [19] E. Nijkamp *et al.*, “CodeGen: An open large language model for code with multi-turn program synthesis.” *arXiv preprint arXiv:2203.13474*, 2022.
- [20] R. Li *et al.*, “StarCoder: May the source be with you!” *arXiv preprint arXiv:2305.06161*, 2023.
- [21] V. Karpukhin *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proceedings of EMNLP 2020*, pp. 6769–6781, 2020.
- [22] M. Lewis *et al.*, “BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” in *Proceedings of ACL 2020*, pp. 7871–7880, 2020.

- [23] M. R. Parvez, W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, “Retrieval augmented code generation and summarization,” in *Findings of the Association for Computational Linguistics: EMNLP 2021*, pp. 2719–2734, 2021.
- [24] F. Zhang *et al.*, “RepoCoder: Repository-level code completion through iterative retrieval and generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 2471–2484, 2023.
- [25] Y. Gao *et al.*, “Retrieval-augmented generation for code comment generation.” arXiv preprint arXiv:2302.08148, 2023.
- [26] J. Feist, G. Grieco, and A. Groce, “Slither: A static analysis framework for smart contracts,” in *2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, pp. 8–15, 2019.
- [27] M. Mossberg *et al.*, “Manticore: A user-friendly symbolic execution framework for binaries and smart contracts,” in *34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 1186–1189, 2019.
- [28] B. Mueller, “Smashing ethereum smart contracts for fun and real profit.” <https://github.com/b-mueller/mythril>, 2018.
- [29] Etherscan, “Ethereum blockchain explorer.” <https://etherscan.io>, 2024.
- [30] OpenZeppelin, “Openzeppelin contracts.” <https://github.com/OpenZeppelin/openzeppelin-contracts>, 2024.
- [31] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [32] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: A method for automatic evaluation of machine translation,” in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 311–318, 2002.
- [33] B. Chen and C. Cherry, “A systematic comparison of smoothing techniques for sentence-level BLEU,” in *Proceedings of the Ninth Workshop on Statistical Machine Translation*, pp. 362–367, 2014.
- [34] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, pp. 74–81, 2004.
- [35] S. Banerjee and A. Lavie, “METEOR: An automatic metric for MT evaluation with improved correlation with human judgments,” in *Proceedings of the ACL*

- Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation*, pp. 65–72, 2005.
- [36] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in *8th International Conference on Learning Representations (ICLR)*, 2020.
- [37] Smart Contract Weakness Classification Registry, “SWC registry.” <https://swcregistry.io>, 2024.
- [38] C. Ferreira Torres *et al.*, “SmartBugs: A framework to analyze solidity smart contracts,” in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 1349–1352, 2020.

Rishav Anand

Turnitin_Plagiarism Report_Rishav Anand_15-05-2026

-  Plagiarism Report
-  Engineering Science & Technology
-  Indian Institute of Technology Patna

Document Details

Submission ID

trn:oid::1:3570641511

Submission Date

May 15, 2026, 4:46 PM GMT+5:30

Download Date

May 15, 2026, 5:42 PM GMT+5:30

File Name

thesis-1.pdf

File Size

1.7 MB

42 Pages

8,456 Words

49,062 Characters

Turnitin_Plagiarism Report_Rishav Anand_15-05-2026

ORIGINALITY REPORT

4%

SIMILARITY INDEX

3%

INTERNET SOURCES

2%

PUBLICATIONS

1%

STUDENT PAPERS

PRIMARY SOURCES

1

arxiv.org

Internet Source

1%

2

dr.ntu.edu.sg

Internet Source

<1%

3

www.iitp.ac.in

Internet Source

<1%

4

www.csse.monash.edu.au

Internet Source

<1%

5

prod-ms-be.lib.mcmaster.ca

Internet Source

<1%

6

psasir.upm.edu.my

Internet Source

<1%

7

github.com

Internet Source

<1%

8

Ji, Zhenlan. "Causality-Driven Interpretation and Inspection of Deep Learning Systems.", Hong Kong University of Science and Technology (Hong Kong)

Publication

<1%

9	Leila Helali. "Efficient Multimodal Generative AI Model Towards Frugal Image Captioning Using Deep Learning and Attention Mechanism", SN Computer Science, 2026 Publication	<1 %
10	Submitted to University of Southampton Student Paper	<1 %
11	d197for5662m48.cloudfront.net Internet Source	<1 %
12	Submitted to Liverpool John Moores University Student Paper	<1 %
13	scholar.sun.ac.za Internet Source	<1 %
14	www.arxiv-vanity.com Internet Source	<1 %

Exclude quotes On
Exclude bibliography On

Exclude assignment template On
Exclude matches < 14 words